

# SageMath: A Final Exploration of Graphs

In the previous articles in this series on SageMath, we explored graph theory extensively and recognised its significance in developing AI-based applications. In this sixth article in the series, we will revisit graph theory concepts due to their growing importance and conclude our discussion on graphs by looking at planar graphs.



Let us continue our discussion of directed graphs by exploring directed cycles. A directed cycle in a directed graph is a closed loop of directed edges. Let us determine whether the graph named *Dgraph\_12*, generated by the SageMath code '*dgraph12.sagews*' discussed in the previous article (*OSFY*, September 2024), contains a directed cycle. To do this, add the following line of code at the end of the '*dgraph12.sagews*' program.

```
DG.is_directed_acyclic( )
```

We use CoCalc to execute SageMath code as before. Upon executing the modified program using CoCalc, we get the output '*False*'. Thus, it is clear that *Dgraph\_12* is cyclic. For example, 2–3–1–2 is a directed cycle of length 3 (see Figure 1 in the previous article).

The discussion of directed cycles naturally leads us to another important type of directed graph: the directed acyclic graph (DAG). DAGs play a pivotal role in AI due to their ability to represent structures where information

flows in a single direction without forming loops. This property is essential for modelling causal relationships and dependencies, making DAGs ideal for applications such as Bayesian networks and various machine learning algorithms. The acyclic nature of DAGs ensures that computations can be carried out efficiently without the risk of infinite loops, making them foundational in AI system design. The program '*dacyclic16.sagews*' shown below generates a directed acyclic graph called *Dag\_16*. Kindly note that programs and graphs discussed in this series are given continuous numbering for ease of reference. The following is the sixteenth complete SageMath program we have discussed in this series.

```
DAG = DiGraph( )
DAG.add_vertices([0, 1, 2, 3, 4, 5])
edges = [(0, 1), (0, 2), (1, 3), (1, 5), (2, 3), (2, 4), (3, 4), (4, 5)]
DAG.add_edges(edges)
DAG.show( )
```